

CHAPITRE 1 : INTRODUCTION AUX COMPILATEURS

1.1 Définition

Un compilateur est un programme qui a comme entrée un code source écrit en langage de haut niveau (langage évolué) et produit comme sortie un code cible en langage de bas niveau (langage d'assemblage ou langage machine).

La traduction ne peut être effectuée que si le code source est correct car, s'il y a des erreurs, le rôle du compilateur se limitera à produire en sortie des messages d'erreurs (voir figure 1.1).

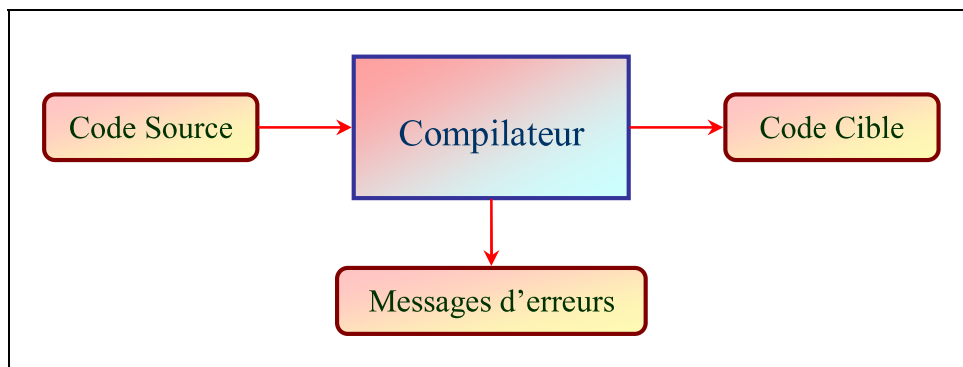


Figure 1.1. Rôle du compilateur

Un compilateur est donc un traducteur de langage évolué qu'on ne doit pas confondre avec un interpréteur qui est un autre type de traducteur. En effet, au lieu de produire un programme cible comme dans le cas d'un compilateur, un interpréteur exécute lui-même au fur et à mesure les opérations spécifiées par le programme source. Il analyse une instruction après l'autre puis l'exécute immédiatement. À l'inverse d'un compilateur, il travaille simultanément sur le programme et sur les données. L'interpréteur doit être présent sur le système à chaque fois que le programme est exécuté, ce qui n'est pas le cas avec un compilateur. Généralement, les interpréteurs sont assez petits, mais le programme est plus lent qu'avec un langage compilé. Un autre inconvénient des interpréteurs est qu'on ne peut pas cacher le code, et donc garder des secrets de fabrication : toute personne ayant accès au programme peut le consulter et le modifier comme elle le veut. Par contre, les langages interprétés sont souvent plus simples à utiliser et tolèrent plus d'erreurs de codage que les langages compilés. Des exemples de langages interprétés sont : BASIC, scheme, CaML, Tcl, LISP, Perl, Prolog

Il existe des langages qui sont à mi-chemin de l'interprétation et de la compilation. On les appelle langages P-code ou langages intermédiaires. Le code source est traduit (compilé) dans une forme binaire compacte (du pseudo-code ou p-code) qui n'est pas encore du code machine. Lorsqu'on exécute le programme, ce P-code est interprété. Par exemple en Java, le programme source est compilé pour obtenir un fichier (.class) « byte code » qui sera interprété par une machine virtuelle. Un autre langage p-code : Python.

Les interpréteurs de p-code peuvent être relativement petits et rapides, si bien que le p-code peut s'exécuter presque aussi rapidement que du binaire compilé. En outre les langages p-code peuvent garder la flexibilité et la puissance des langages interprétés.

1.2 Structure générale d'un compilateur

Un compilateur est généralement composé de modules correspondant aux phases logiques de l'opération de compilation (voir figure 1.2).

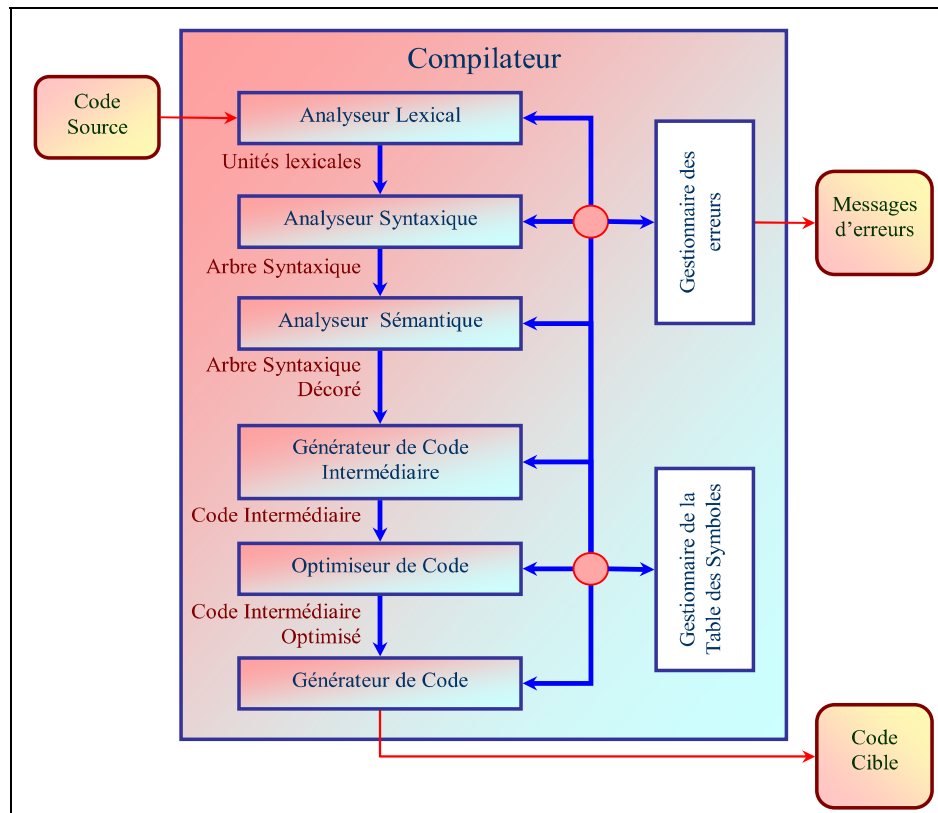


Figure 1.2. Phases et modules de compilation

Chacun des modules de la figure 1.2 (excepté les modules gestionnaires de table de symboles et d'erreurs), représente une phase logique qui reçoit en entrée une représentation de code source et la transforme en une autre forme de représentation.

La structure représentée par la figure 1.2 est purement conceptuelle. Elle correspond à l'organisation logique d'un compilateur. En pratique, plusieurs phases peuvent être regroupées en une seule passe qui reçoit en entrée une certaine représentation et donne en sortie une autre.

Par exemple, les phases d'analyses lexicale, syntaxique, sémantique et la génération du code intermédiaire peuvent correspondre à seule passe dans laquelle l'analyseur syntaxique est le module maître qui appelle, à chaque fois, l'analyseur lexical pour obtenir une unité lexicale, puis déterminer graduellement la structure syntaxique du code et enfin appelle le générateur de code qui effectue l'analyse sémantique et produit une partie du code intermédiaire.

1.2.1 L'analyseur lexical

Connu aussi sous l'appellation **Scanner**, l'analyseur lexical a pour rôle principal la lecture du texte du code source (suite de caractères) puis la formation des unités lexicales (appelées aussi entités lexicales, lexèmes, jetons, tokens ou encore atomes lexicaux).

Exemple

Considérons l'expression d'affectation **a := b + 2 * c ;**

Les unités lexicales qui apparaissent dans cette expression sont :

Unité lexicale	Sa nature
a	Identificateur de variable
:=	Symbole d'affectation
b	Identificateur de variable
+	Opérateur d'addition
2	Valeur entière
*	Opérateur de multiplication
c	Identificateur de variable
;	Séparateur

L'analyseur a aussi comme rôle l'élimination des informations inutiles pour l'obtention du code cible, le stockage des identificateurs dans la table des symboles et la transmission d'une entrée à l'analyseur syntaxique. Concernant les informations inutiles, il s'agit généralement du caractère espace et des commentaires.

1.2.2 L'analyseur syntaxique

L'analyseur syntaxique (appelé Parser en anglais) a pour rôle principal la vérification de la syntaxe du code en regroupant les unités lexicales suivant des structures grammaticales qui permettent de construire une représentation syntaxique du code source. Cette dernière a souvent une structure en arbre. Notons que durant cette phase, des informations, telles que le type des identificateurs, sont enregistrées dans la table des symboles

Exemple

La représentation sous forme d'arbre syntaxique de l'expression « **a := b + 2 * c ;** » est donnée par la figure 1.3. Dans cette structure d'arbre, les nœuds représentent des opérateurs et les feuilles de l'arbre représentent les valeurs et les variables sur lesquelles s'effectuent les opérations. La figure donne aussi le parcours qui est fait sur cet arbre lors de l'évaluation. D'autres parcours peuvent être envisagés pour réaliser différentes tâches, cependant ces parcours ont lieu dans les phases ultérieures de la compilation.

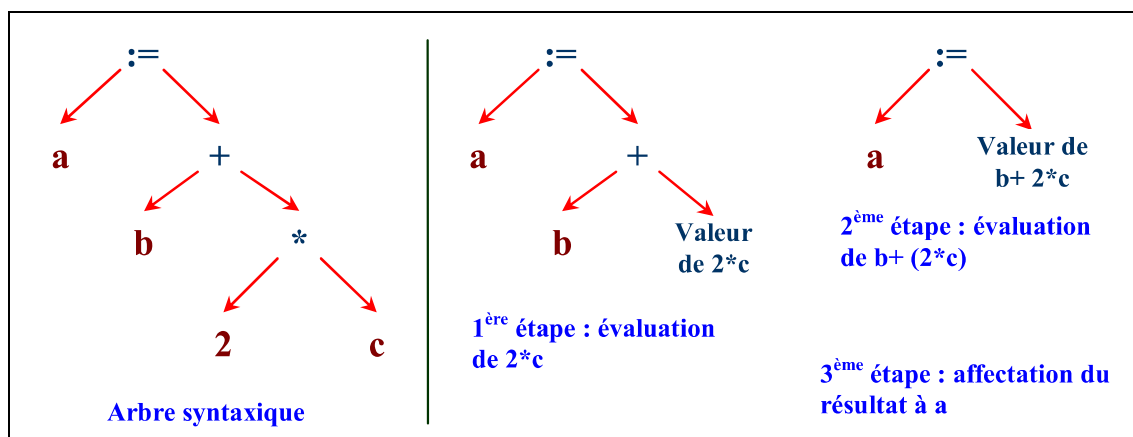


Figure 1.3. Arbre syntaxique et parcours d'évaluation

1.2.3 L'analyseur sémantique

Il a comme rôle principal le contrôle du code source, pour détecter éventuellement l'existence d'erreurs sémantiques, et la collecte des informations destinées à la production du code intermédiaire. Un des constituants importants de la phase d'analyse sémantique est le contrôle du type qui consiste à vérifier si les opérandes de chaque opérateur sont conformes aux spécifications du langage utilisé pour le code source.

Exemple

Dans l'analyse sémantique de « **a := b + 2 * c ;** », il faut vérifier que, si a est de type entier, alors b et c le sont aussi, sinon il faut signaler une erreur.

Si on suppose que a, b et c sont de type réel, alors pendant l'évaluation de l'expression, l'analyse sémantique aura comme tâche d'insérer une opération de conversion de type pour transformer la valeur entière 2 en valeur réelle 2.0. Cela peut être effectué sur l'arbre syntaxique comme le montre la figure 1.4 (arbre syntaxique décoré).

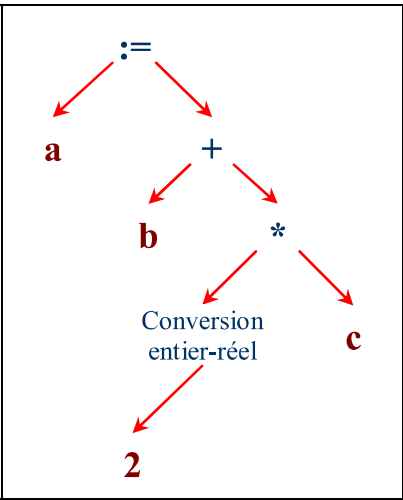


Figure 1.4. Enrichissement de l'arbre syntaxique lors de la phase d'analyse sémantique

1.2.4 Le générateur de code intermédiaire

Certains compilateurs construisent explicitement une représentation intermédiaire du code source sous forme d'un code intermédiaire qui n'est pas directement exécutable par une machine spécifique. C'est plutôt un code généré pour une machine abstraite (virtuelle), qui a la double caractéristique d'être, à la fois, facile à produire, à partir de l'arbre syntaxique, et facile à convertir pour une machine réelle donnée.

Exemple

Nous avons supposé que la machine abstraite est une machine à une adresse qui dispose d'un seul accumulateur et dont le jeu d'instruction contient les instructions LoadValue, ConvReal, Mul, Add et Store. Elles permettent de réaliser les opérations données dans la deuxième colonne. Nous avons supposé aussi que **a** occupe la première entrée dans la table des symboles, **b** occupe la deuxième et **c** la troisième.

Ainsi, le code intermédiaire de l'expression « **a := b + 2 * c ;** », peut être comme suit :

Code	Signification opérationnelle des instructions
LoadValue 2	Charger l'accumulateur avec une valeur directe (2)
ConvReal	Convertir le contenu de l'accumulateur en réel
Store temp1	Stocker le résultat dans la variable temporaire temp1
Load temp1	Charger l'accumulateur avec la valeur de temp1
Mul 3	Multiplier le contenu de l'accumulateur par le contenu de l'entrée 3 de la table de symboles (c)
Add 2	Additionner le contenu de l'accumulateur au contenu l'entrée 2 de la table de symboles (b)
Store 1	Ranger le contenu de l'accumulateur dans l'entrée 1 de la table de symboles (a)

1.2.5 L'optimiseur du code intermédiaire

Lors de la phase d'optimisation, le code intermédiaire est changé pour améliorer les performances du code cible qui en sera généré. Il s'agit principalement de réduire le temps d'exécution et l'espace mémoire qui sera occupé par le code cible. L'optimisation supprime, par exemple, les identificateurs non utilisés, élimine les instructions inaccessibles, élimine les instructions non nécessaires, fait ressortir hors des boucles les instructions qui ne dépendent pas de l'indice de parcours des boucles, etc.

L'optimisation risque de ralentir le processus de compilation dans son ensemble mais elle peut avoir un effet positif considérable sur le code cible qui sera généré ultérieurement.

Exemple

On constate dans l'exemple précédent que la valeur convertie 2.0 est stockée dans temp1 puis récupérée et chargée dans l'accumulateur. Puisque aucun usage n'est fait de temp1 dans le reste du code, il est possible d'éliminer les deux instructions en question. Après conversion, le résultat 2.0 reste alors dans l'accumulateur et sera utilisé directement dans la multiplication. Noter que, à l'opposé, si la conversion en réel de la valeur 2 est souvent nécessaire, il serait préférable de la stocker dans un espace temporaire. Cependant, ce dernier augmente l'espace réservé aux données dans le code cible.

Le nouveau code intermédiaire est le suivant :

Code	Signification opérationnelle des instructions
LoadValue 2	Charger l'accumulateur avec une valeur directe
ConvReal	Convertir le contenu de l'accumulateur en réel
Mul 3	Multiplier le contenu de l'accumulateur par le contenu de l'entrée 3 de la table de symboles
Add 2	Additionner le contenu de l'accumulateur au contenu l'entrée 2 de la table de symboles
Store 1	Ranger le contenu de l'accumulateur dans l'entrée 1 de la table de symboles

1.2.6 Le générateur du code cible

C'est la phase finale d'un compilateur qui consiste à produire du code cible dans un langage d'assemblage ou un langage machine donné. Le code généré est directement exécuté par la machine en question ou alors il l'est après une phase d'assemblage.

Exemple

Considérons une machine à deux adresses qui dispose de deux registres de calcul R1 et R2. Nous supposons que les variables a, b et c de la table des symboles ont comme adresses de cellules mémoires correspondantes ad1, ad2 et ad3. Le code cible qui sera généré pour cette machine est donné dans le tableau suivant :

Code	Signification opérationnelle des instructions
MOV R1, #2	Charger R1 avec la valeur directe 2
CReal R1	Convertir le contenu de R1 en réel
MOV R2, ad3	Charger le registre R2 avec le contenu de la cellule mémoire d'adresse ad3
MUL R1, R2	Multiplier le contenu de R1 par le contenu de R2, le résultat est dans le premier registre
MOV R2, ad2	Charger R2 avec le contenu de l'adresse ad2
ADD R1, R2	Additionner le contenu de R1 au contenu de R2, le résultat est dans le premier registre
STO R1, ad1	Ranger le contenu de R1 dans la cellule mémoire d'adresse ad1

1.2.7 Le gestionnaire de la table de symbole

Les phases logiques de compilation échangent des informations par l'intermédiaire de la table des symboles. C'est une structure de données (généralement une table) contenant un enregistrement pour chaque identificateur utilisé dans le code source en cours d'analyse. L'enregistrement contient, parmi d'autres informations, le nom de l'identificateur, son type, et l'emplacement mémoire qui lui correspondra lors de l'exécution.

A chaque fois que l'analyseur lexical rencontre un identificateur pour la première fois, le gestionnaire de la table des symboles insère un enregistrement dans la table et l'initialise avec les informations actuellement disponibles (le nom). Lors de l'analyse syntaxique, le gestionnaire associera le type à l'identificateur, alors que, lors de l'analyse sémantique, une vérification de types est opérée grâce à cet enregistrement.

1.2.8 Le gestionnaire des erreurs

Son rôle est de signaler les erreurs qui peuvent exister dans le code source et qui sont détectées lors des différentes phases logiques de la compilation. Il doit produire, pour chaque erreur, un diagnostic clair et sans ambiguïté qui permettra la localisation et la correction de l'erreur par l'auteur du code source.

1.3 Outils de construction des compilateurs

Suite au développement des premiers compilateurs, on s'est très vite rendu compte que certaines tâches liées au processus de compilation peuvent être automatisées, ce qui facilite grandement la construction des compilateurs. La notion d'outils de construction de compilateurs est alors apparue.

La première catégorie est représentée par des outils généraux appelés Compilateurs de Compilateurs, Générateurs de Compilateurs ou Systèmes d'écriture de traducteurs. Les outils de cette catégorie se sont souvent orientés vers un modèle particulier de langages et ne sont adaptés qu'à la construction de compilateurs pour des langages correspondant à ce modèle.

La deuxième catégorie correspond à des outils spécialisés dans la construction automatique de certaines phases d'un compilateur, tels que :

- Les constructeurs ou générateurs automatiques d'analyseurs lexicaux à partir d'une spécification contenant des expressions régulières
- Constructeurs ou générateurs automatiques d'analyseurs syntaxiques à partir d'une spécification basée sur une grammaire non contextuelle et en utilisant des algorithmes d'analyse puissants mais difficile à mettre en œuvre manuellement.

CHAPITRE 2 : ANALYSE LEXICALE

2.1 Rôle de l'analyseur lexical

Il s'agit de transformer des suites de caractères du code source en suite de symboles, correspondant aux unités lexicales, que l'analyseur lexical produit comme résultat de l'analyse. Pour cela, il existe deux approches possibles sachant qu'une passe est définie comme correspondant à un traitement entier d'une représentation du programme source pour produire une représentation équivalente en mémoire secondaire :

- L'analyseur lexical effectue un traitement de la totalité du code source en une première passe, ce qui lui permet d'en obtenir une représentation équivalente sous forme d'une suite d'unités lexicales, sauvegardée dans un fichier en mémoire secondaire. Ce fichier sera l'entrée de l'analyseur syntaxique, qui accomplira son travail pendant une deuxième passe. Le schéma de la figure 2.1 illustre cette approche.
- L'analyseur lexical fonctionne comme une procédure sollicitée à chaque fois, par l'analyseur syntaxique, pour lui fournir une unité lexicale. L'analyseur lexical effectue, pour cela, son travail, pas à pas, en synchronisation avec l'avancement de l'analyse syntaxique. On dit que l'analyse lexicale et syntaxique partagent une même passe. Dans ce cas, il n'est plus question de sauvegarder les unités dans un fichier intermédiaire, cependant, les deux analyseurs (lexical et syntaxique) doivent se trouver ensemble en mémoire. Cette approche est la plus utilisée dans la conception des compilateurs, elle est illustrée par la figure 2.2.

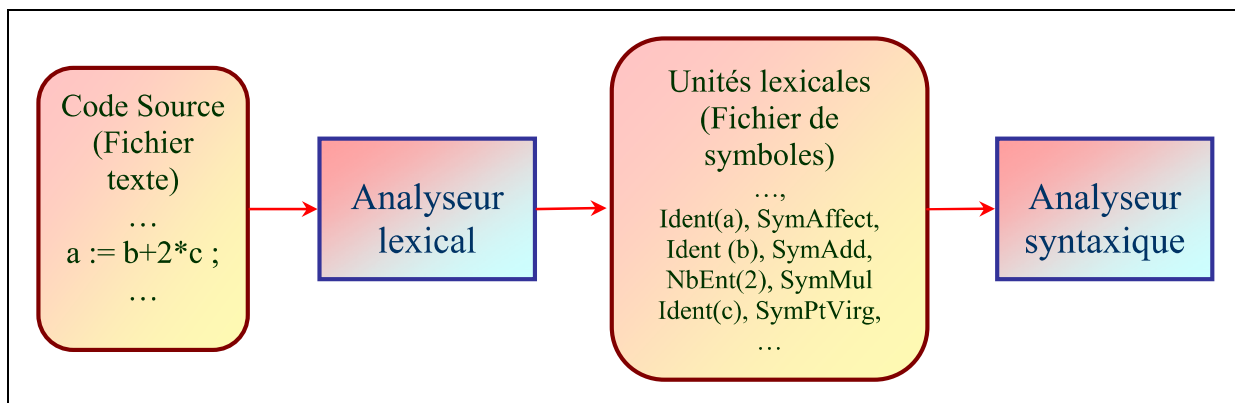


Figure 2.1. Analyse lexicale et syntaxique en deux passes différentes

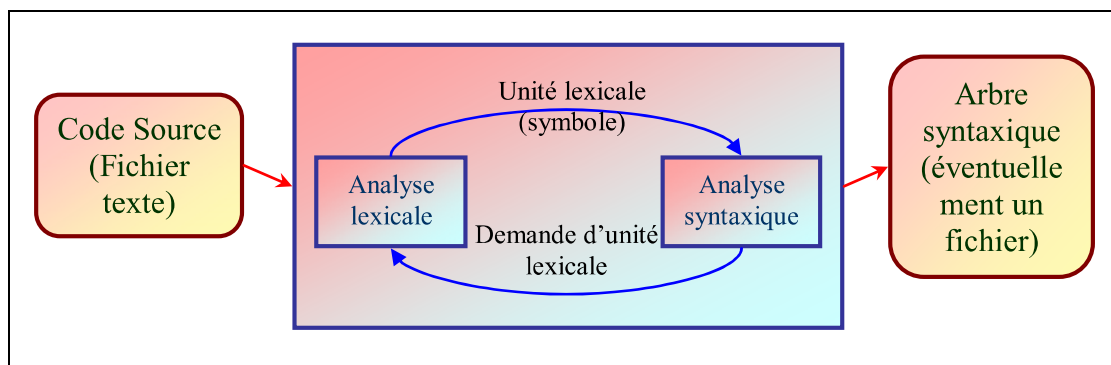


Figure 2.2. Analyses lexicale et syntaxique en une seule passe

2.2 Tâches effectuées par l'analyseur lexical

L'analyse lexicale effectue un ensemble de tâches :

- 1- Lecture du fichier du code source (fichier texte) caractère par caractère avec éventuellement l'élimination de caractères et informations inutiles (blancs, tabulations, commentaires, caractère de fin de ligne, ...) pour réduire la représentation du programme. Une erreur est signalée à ce niveau si un caractère non permis (illégal) par le langage est rencontré.

- 2- Concaténation des caractères pour former des unités lexicales et signaler, éventuellement, une erreur si la chaîne dépasse une certaine taille.
- 3- Association d'un symbole à chaque unité lexicale. Cette opération peut engendrer une erreur si l'unité formée ne correspond à aucune unité légale du langage.
- 4- Enregistrement de chaque unité lexicale et des informations la concernant (nom, valeur, ...) dans la table des symboles (en invoquant le gestionnaire de la table des symboles) ou dans un fichier résultat. A ce niveau, il est possible de détecter certaines erreurs telles que la double déclaration d'un identificateur, l'utilisation d'un identificateur sans déclaration préalable, etc.
- 5- Création d'une liaison entre les unités lexicales d'une ligne et la ligne du fichier la contenant. Cette opération, qui se fait par une simple numérotation des lignes du texte du code source, permet la localisation des lignes en cas d'erreur. Dans certains compilateurs, l'analyseur lexical est chargé de créer une copie du programme source en y intégrant les messages d'erreurs lexicales.

Les symboles associés aux unités lexicales correspondent à la nature de celles-ci. On distingue plusieurs catégories de symboles :

- 1- Les mots réservés du langage (if, then, begin pour le langage Pascal, par exemple)
- 2- Les constantes (3, 3.14, True, 'Bonjour', ...)
- 3- Les identificateurs qui peuvent être des noms de variables, de fonctions, de procédures, etc.
- 4- Les symboles spéciaux, en tenant compte des possibilités d'assemblage de caractères tels que < > <= > : := + - /*
- 5- Les séparateurs tels que ; (point virgule) et . (point).

2.3 Spécification des unités lexicales

L'ensemble des unités lexicales, qu'un analyseur reconnaît, constitue un langage régulier L qui peut être décrit par des expressions régulières et reconnu par un automate d'états fini.

La spécification ou description des unités lexicales peut donc être effectuée en utilisant une notation appelée **expressions régulières**. Une expression régulière est construite à partir d'expressions régulières plus simples, en utilisant un ensemble de règles de définition qui spécifient comment le langage est formé.

Soit le vocabulaire $\Sigma = \{a, b, c\}$

- 1- ε dénote la chaîne vide
- 2- L'expression régulière a/b dénote l'ensemble des chaînes $\{a, b\}$
- 3- L'expression a^* dénote l'ensemble de toutes les chaînes formées d'un nombre quelconque de a (pouvant être nul), c à $d \{\varepsilon, a, aa, aaa, \dots\}$
- 4- L'expression a^+ dénote l'ensemble de toutes les chaînes formées d'un nombre quelconque, non nul, de a , c à $d \{a, aa, aaa, \dots\}$. Si r est une expression régulière alors $r^* = r^+/\varepsilon$ et $r^+ = rr^*$
- 5- La notation $r?$ est une abréviation de r/ε et signifie zéro ou une instance de r
- 6- La notation $[abc]$, où a, b, c sont des symboles de l'alphabet, dénote l'expression régulière $a/b/c$. Une classe de caractères $[a-z]$ signifie $a/b/c/d/ \dots/z$

Exemple 1

Pour chacune des expressions régulières r_i suivantes, on souhaite déterminer le langage dénoté par r_i .

$$\begin{aligned} r_1 &= (a/b)(a/b) \\ r_2 &= (a/b)^* \\ r_3 &= (a^*b^*)^* \\ r_4 &= a/a^*b \end{aligned}$$

Chaque langage dénoté par r_i est noté $L(r_i)$

$$L(r_1) = \{aa, ab, ba, bb\}$$

$L(r_2) = \{\varepsilon, a, aa, aaa, \dots, b, bb, bbb, \dots, ab, aab, \dots, abb, abbb, ba, bba, bbaa, aba, \dots\}$. C'est l'ensemble de toutes les chaînes composées d'un nombre quelconque de a et d'un nombre quelconque de b .

$$L(r3) = L(r2)$$

$L(r4) = \{a, b, ab, aab, aaab, \dots, aaaa\dots ab\}$ En plus de la chaîne a, cet ensemble contient des chaînes composées d'un nombre quelconque non nul de a suivis par un b.

Exemple 2

On souhaite déterminer la définition régulière de l'ensemble des identificateurs d'un langage sachant qu'ils sont constitués de suite de lettres et de chiffres commençant par une lettre.

Cette définition régulière peut être donnée par :

- lettre = A/B/.../Z/a/b/.../z ou bien [A-Za-z]
- chiffre = 0/1/2/3/4/5/6/7/8/9 ou bien [0-9]
- ident = lettre (lettre/chiffre)* qu'on peut noter directement ident = [A-Za-z] [A-Za-z0-9]*

Remarque

Les expressions régulières sont un outil puissant et pratique pour définir les constituants élémentaires des langages de programmation. Cependant, le pouvoir descriptif des expressions régulières est limité car elles ne peuvent pas être utilisées dans certains cas tels que :

- ❖ La définition des constructions équilibrées ou imbriquées. Par exemple, pour les chaînes contenant obligatoirement pour chaque parenthèse ouvrante une parenthèse fermante, les expressions régulières ne permettent pas d'assurer cela, on dit qu'elles **ne savent pas compter**.
- ❖ La définition d'expressions contenant des répétitions de sous chaînes à différentes positions (par exemple des chaînes de la forme $\alpha\alpha\alpha\alpha$ où α est une combinaison des caractères a et b).
- ❖ La définition d'expressions contenant à différentes positions des séquences de caractères ayant la même longueur (par exemple $a^n b^n$ où n désigne le nombre de répétitions du caractère qui le précède).

On peut, cependant, utiliser les expressions régulières dans le cas d'un nombre **fixe** de répétitions d'une construction donnée.

2.4 Approches de construction des analyseurs lexicaux

Après la description des unités lexicales, la construction de l'analyseur lexical peut être effectuée selon l'une des manières suivantes :

- 1- Une approche simplifiée (manuelle) basée sur les diagrammes de transition.
- 2- Une approche plus rigoureuse basée sur les automates d'états finis.
- 3- Une approche utilisant un outil générateur d'analyseurs lexicaux tel que Lex, FLex, JFLex...

Dans les sections suivantes nous décrivons chacune des trois approches.

2.5 Construction d'analyseur lexical basée sur les diagrammes de transition

Les diagrammes de transition sont des organigrammes stylisés (respectant une forme stricte) dont la construction représente une étape préparatoire pour la réalisation d'un analyseur lexical. Le diagramme est ensuite converti en un programme qui permet de reconnaître les unités lexicales exprimées par le diagramme.

2.5.1 Introduction aux diagrammes de transitions

Formellement, un diagramme de transition est un diagramme contenant des cercles numérotés qui représentent des états et des arcs étiquetés matérialisant les transitions entre les états.

Les états du diagramme de transition correspondent aux états de l'analyseur lexical et chaque arc étiqueté par un caractère donné correspond à la transition qu'effectuera l'analyseur suite à la rencontre de ce caractère. Ainsi, au fur et à mesure de la reconnaissance d'une unité lexicale, l'analyseur passe d'un état à un autre.

Considérons deux états e1 et e2 reliés par un arc étiqueté par le caractère c. Etant dans l'état e1 à un moment donné, l'analyseur passera à l'état e2 lorsqu'il lit le caractère c dans le texte source.

Nous adopterons les notations suivantes dans la suite de cette section.

- Un état particulier du diagramme représentera l'état initial où commence la reconnaissance d'une unité lexicale. Il est noté par un état auquel abouti un arc étiqueté par **début**.
- Les états finaux sont identifiés par deux cercles concentriques (cercle doublé) et correspondent, chacun, à la reconnaissance complète d'une unité lexicale.
- Si un ensemble d'arcs sortant d'un état e1 comprend un arc ayant l'étiquette **autre**, elle désigne tous les caractères autres que ceux indiqués explicitement sur les autres arcs sortant de e1.

Remarque

On suppose que les diagrammes de transition utilisés sont **déterministes**. En d'autres termes, aucun caractère ne peut apparaître comme étiquette de deux ou plusieurs arcs quittant un même état.

Exemple 1

Le diagramme de transition de la figure 2.3, correspond à l'expression régulière `lettre(lettre/chiffre)*` et permet de représenter les identificateurs utilisés dans les langages tel que Pascal (voir exemple 2, section 2.3).

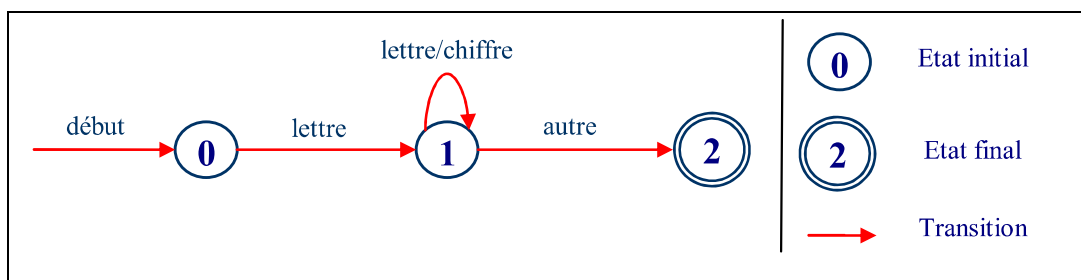


Figure 2.3. Diagramme de transition des identificateurs

Exemple 2

On souhaite donner la définition régulière ainsi que le diagramme de transition qui représente l'ensemble des nombres réels non signés comme 2006, 12.33, 314.3E-2, 0.314E+1, 0.314E2, 314E-2

La définition régulière est donnée par :

- chiffre = 0/1/2/3/4/5/6/7/8/9 ou bien [0-9]
- chiffres = chiffre chiffre* ou bien chiffre+
- fractionOpt = .chiffres/ε ou bien (.chiffres)?
- exposantOpt = (E(+/-ε) chiffres) / ε ou bien (E(+/-)?chiffres)?
- NbRéal = chiffres fractionOpt exposantOpt

Pour faciliter la représentation par un diagramme de transition, la spécification précédente peut être réécrite directement sous la forme :

$\text{NbRéal} = \text{chiffre chiffre}^* (. \text{chiffre chiffre}^*)? (E(+/-)?\text{chiffre chiffre}^*)?$

Cette forme peut être représentée directement de la figure 2.4.

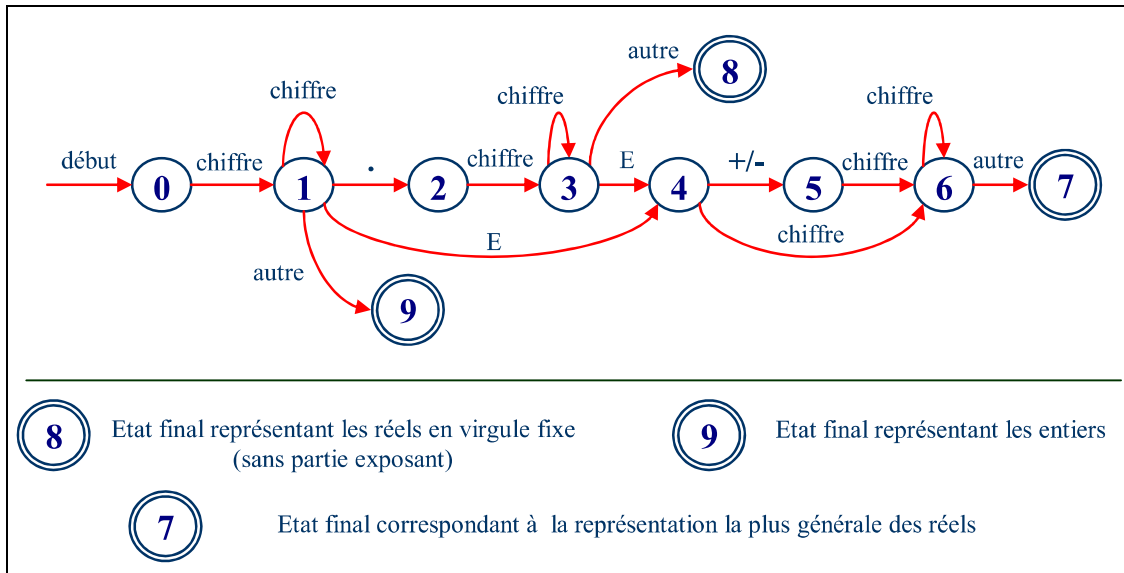


Figure 2.4. Diagramme de transition des nombres réels

2.5.2 Exemple d'implémentation d'un diagramme de transition

L'implémentation du diagramme de transition d'un identificateur (voir figure 2.3) peut être décrite comme suit. Nous utiliserons, pour des raisons de simplification, une série de procédures et fonctions que nous décrivons en premier.

- **lettre** est une fonction qui retourne vrai si le caractère courant passé dans ch est un caractère alphabétique.
- **carsuiv** est une procédure qui retourne le caractère suivant du texte source dans ch (passé en paramètre).
- **retournerId** permet de retourner l'identificateur reconnu. Pour cela, elle associe le symbole correspondant (id) à l'unité formée, range l'identificateur dans la table des symboles, (s'il n'y est pas déjà) et retourne son adresse.
- **chiffre** est une fonction qui retourne vrai si le caractère ch (passé en paramètre) est un chiffre.
- **échec** est une procédure qui permet de passer à un autre diagramme de transition (s'il y en a) ou d'appeler la procédure **erreur**.

Fonction lettre (ch : caractère) : booléen ;

Début

Si (ch ≥ 'A' et ch ≤ 'Z') **ou** (ch ≥ 'a' et ch ≤ 'z') **Alors** lettre ← Vrai

Sinon lettre ← Faux ;

Fin ;

Fonction chiffre (ch : caractère) : booléen ;

Début

Si (ch ≥ '0' Et ch ≤ '9') **Alors** chiffre ← Vrai

Sinon chiffre ← Faux ;

Fin ;

Etapes de l'implémentation

Etat₀ : carsuiv (ch) ;

Si lettre(ch) alors Aller à état₁

Sinon échec() ;

Etat₁ : carsuiv (ch) ;

Si lettre(ch) Ou chiffre(ch) alors Aller à état₁

Sinon Aller à état₂ ;

Etat₂ : retournerId()

2.5.3 Démarche générale d'implémentation d'un analyseur lexical à partir d'un diagramme de transition

D'une manière plus générale, une suite de diagrammes de transition peut être convertie en un programme qui recherche les unités lexicales selon l'approche suivante :

- Chaque état donne lieu à un segment de programme.
- S'il y a des arcs qui quittent un état, son segment de programme doit inclure la lecture d'un caractère et son test pour sélectionner un arc à suivre.
- S'il y a un arc étiqueté par le caractère lu, ou par une classe de caractères contenant le caractère lu, alors le contrôle est transféré au code correspondant à l'état pointé par cet arc.
- S'il n'existe pas un arc correspondant à la valeur de ch et si l'état actuel n'appartient pas à l'ensemble des états qui indiquent la rencontre d'une unité lexicale (i.e. ce n'est pas un état d'acceptation), alors il y a un échec de l'analyse et la recherche est initialisée à celle de l'unité lexicale spécifiée par le prochain diagramme de transition.
- On utilise généralement une instruction de choix multiple pour trouver l'état de départ du diagramme de transition suivant. On suit les arcs du diagramme de transition en sélectionnant le fragment de code d'un état et on exécute ce fragment pour déterminer le prochain état.

2.6 Construction d'analyseur lexical basée sur les automates finis

2.6.1 Démarche générale de construction d'un analyseur lexical

En se basant sur les propositions démontrables suivantes :

- L'ensemble des unités lexicales d'un langage donné constitue un langage régulier L.
- Pour toute expression régulière r, il existe un automate fini non déterministe (AFN) qui accepte l'ensemble régulier décrit par r.
- Si un langage L est accepté par un automate fini non déterministe (AFN), alors il existe automate fini déterministe (AFD) acceptant L.

On peut définir une approche rigoureuse pour la construction d'un analyseur lexical en utilisant les automates d'états finis. Cette approche est constituée de 6 étapes :

Etape 1 : Spécification des unités lexicales

Spécifier chaque type d'unité lexicale à l'aide d'une expression régulière (ER).

Etape 2 : Conversion ER en AFN

Convertir chaque expression régulière en un automate d'états fini (non déterministe).

Etape 3 : Réunion des AFNs

Construire l'automate Union de tous les automates de l'étape 2

(on peut ajouter un nouvel état initial d'où partent un ensemble d'arcs étiquetés ϵ).

Etape 4 : Déterminisation ou transformation de l'AFN obtenu en AFD

Rendre l'automate de l'étape 3 déterministe.

Etape 5 : Minimisation de l'AFD.

Minimiser l'automate obtenu à l'étape 4.

Etape 6 : Implémentation de l'AFD minimisé.

Implémenter l'automate obtenu à l'étape 5 en le simulant à partir de sa table de transition.

2.6.2 Automates à états finis

On transforme une expression régulière en un reconnaiseur en construisant un diagramme de transition généralisé appelé automate à états finis. Ce dernier peut être déterministe (AFD) ou non déterministe (AFN).

	AFN	AFD
Temps reconnaissance	Augmente (à cause des retours arrière)	Diminue (un seul chemin possible pour une chaîne donnée)
Espace occupé	Nombre d'états plus petit que celui de l'AFD équivalent	Nombre d'états généralement plus grand que celui de l'AFN équivalent

La transformation d'un automate à états finis en un analyseur lexical consiste à associer une unité lexicale à chaque état final et à faire en sorte que l'acceptation d'une chaîne produise comme résultat l'unité lexicale associée à l'état final en question. Pour cela il faut implémenter la fonction de transition de l'automate en utilisant une table de transition.

2.6.2.1 Automate fini non déterministe

Un AFN est défini par :

- Un ensemble d'état E
- Un ensemble de symboles d'entrée ou alphabet Σ
- Un état initial e_0
- Un ensemble F d'états finaux ou d'acceptation
- Une fonction de transition *Transiter* qui fait correspondre à chaque couple (état,symbole), un ensemble d'états.

Exemple

Considérons l'AFN qui reconnaît le langage décrit par l'expression régulière $(a|b)^*abb$ (voir figure 2.5). Pour cet automate :

$$E = \{0, 1, 2, 3\} \quad e_0 = 0 \quad \Sigma = \{a, b\} \quad F = \{3\}$$

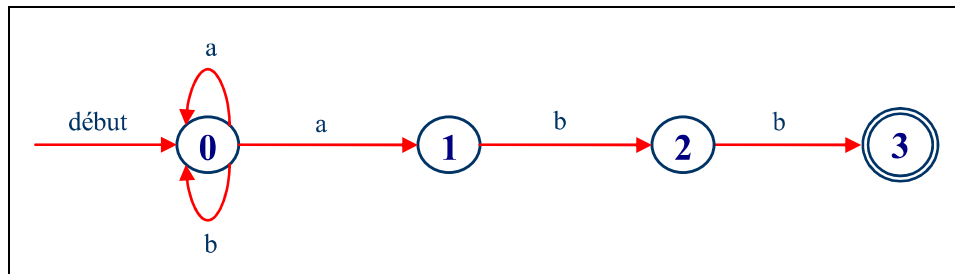


Figure 2.5. Un AFN pour l'expression $(a|b)^*abb$

L'implémentation la plus simple d'un AFN est une table de transition dans laquelle il y a une ligne pour chaque état et une colonne pour chaque symbole d'entrée (avec la chaîne vide ϵ si nécessaire). L'entrée pour la ligne i et le symbole a donne l'ensemble des états (ou un pointeur vers cet ensemble) qui peuvent être atteints à partir de l'état i avec le symbole a . Pour l'AFN de la figure 2.5, la table de transition est la suivante (sachant que la ligne 3 peut être éliminée) :

Etats	Symboles	
	a	b
0	{0, 1}	{0}
1	/	{2}
2	/	{3}
3	/	/

Un AFN accepte une chaîne si et seulement s'il existe au moins un chemin correspondant à cette chaîne entre l'état initial et l'un des états finaux. Un chemin peut être représenté par une suite de transitions appelées *déplacements*. Par exemple, pour la reconnaissance de la chaîne $aabb$ par l'AFN de la figure 2.5, il existe deux chemins possibles. Le premier conduit à l'acceptation de la chaîne alors que le second stationne sur l'état 0 qui n'est pas un état final (voir figures 2.6 et 2.7).

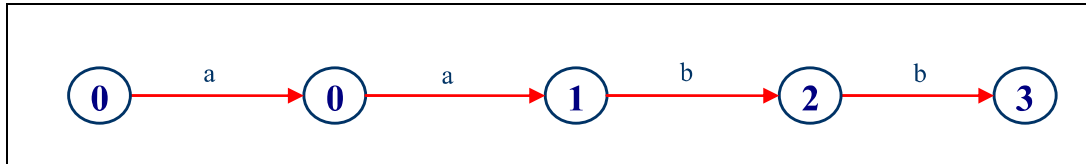


Figure 2.6. Déplacements réalisés en acceptant la chaîne aabb

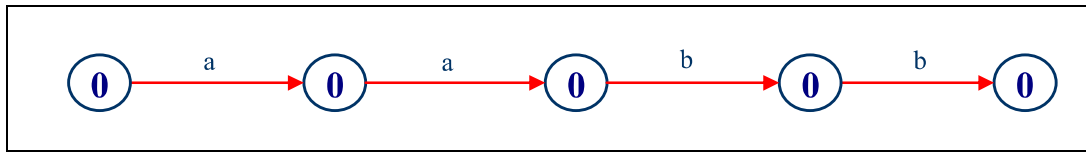


Figure 2.7. Stationnement sur l'état 0 pour la chaîne aabb

2.6.2.2 Automate fini déterministe

Un AFD est un cas particulier d'AFN dans lequel :

- Aucun état n'a de ϵ -transition
- Pour chaque état **e** et chaque symbole d'entrée **a** il y a au plus un arc étiqueté **a** qui quitte **e**

Dans la table de transition d'un AFD, une entrée contient un état unique au maximum (les symboles d'entrée sont les caractères du texte source), il est donc très facile de déterminer si une chaîne est acceptée par l'automate vu qu'il n'existe, au plus, qu'un seul chemin entre l'état initial et un état final étiqueté par la chaîne en question.

Remarque : La table de transition d'un AFN pour un modèle d'expression régulière peut être considérablement plus petite que celle d'un AFD. Cependant, l'AFD présente l'avantage de pouvoir reconnaître des modèles d'expressions régulières plus rapidement que l'AFN équivalent.

2.6.3 Construction d'AFN à partir d'expressions régulières (étape 2)

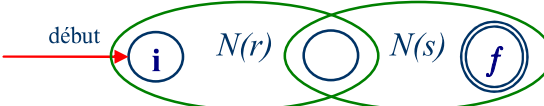
Il existe plusieurs algorithmes pour effectuer cette construction. Parmi les plus simples et les plus faciles à implémenter on cite l'algorithme de construction de **Thompson**. Dans la suite, nous utiliserons les notations suivantes :

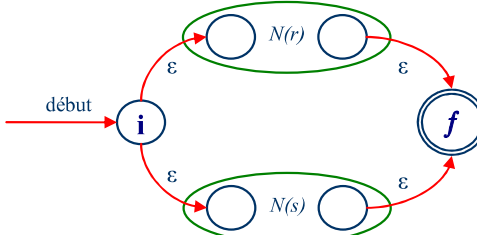
- r : désigne une expression régulière
- $N(r)$: l'AFN correspondant à r (qui reconnaît r)

Les règles de l'algorithme de Thompson sont les suivantes :

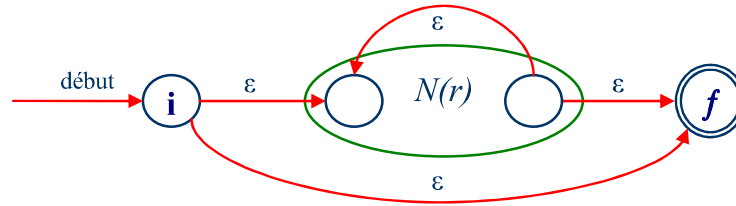
1. Pour $r = \epsilon$ $N(\epsilon)$ est représenté par: 

2. Pour $r = a$ $N(a)$ est représenté par: 

3. Pour $N(rs)$ on a : 

4. Pour $N(r/s)$ on a : 

5. Pour r^* , $N(r^*)$ sera donnée par la représentation suivante :



Exemple

L'AFN qui reconnaît l'expression régulière $(a|b)^*abb$ selon la méthode de construction de Thompson est donné dans la figure 2.8 :

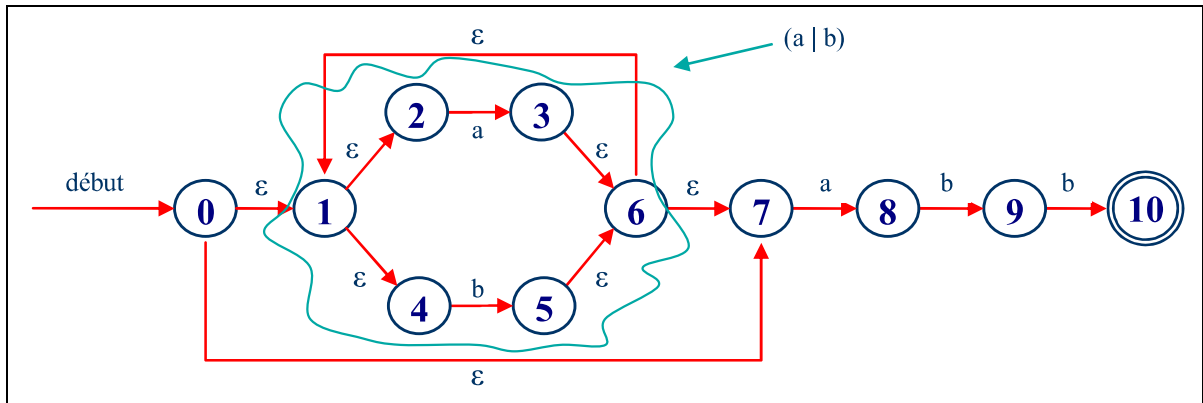


Figure 2.8. Automate fini non déterministe N pour accepter $(a|b)^*abb$

2.6.4 Détermination d'un AFN (étape 4)

La détermination consiste à transformer un AFN en un AFD équivalent. Elle peut être effectuée par différentes méthodes, nous en présentons trois dans ce qui suit.

2.6.4.1 Détermination d'un AFN ne contenant pas de ϵ -transitions

La détermination d'un AFN ne contenant pas de ϵ -transitions peut être effectuée en appliquant l'algorithme suivant sur la table de transition de l'AFN pour en déduire celle de l'AFD équivalent :

Etapas de l'algorithme

- 1- Partir de l'état initial $E_0 = \{e_0\}$
- 2- Construire E_1 qui est l'ensemble des états obtenus à partir de E_0 par la transition étiquetée **a**, $E_1 = \text{Transiter}(E_0, a)$.
- 3- Recommencer l'étape 2 pour toutes les transitions possibles et pour chaque nouvel ensemble d'états E_i .
- 4- Tous les ensembles d'états E_i contenant au moins un état final deviennent finaux.
- 5- Renommer les ensembles d'états obtenus en tant que simples états.

Exemple

Appliquons l'algorithme précédent sur l'AFN dont $e_0 = 0$ et $F = \{2, 3\}$ et ayant la table de transition suivante :

Etats	Symboles	
	a	b
0	{0, 2}	{1}
1	{3}	{0, 2}
2	{3, 4}	{2}
3	{2}	{1}
4	/	{3}

Après application de l'algorithme, nous obtenons la table de transition de l'AFD suivante :

Etats	Symboles	
	a	b
{0}	{0, 2}	{1}
{0, 2}	{0, 2, 3, 4}	{1, 2}
{1}	{3}	{0, 2}
{0, 2, 3, 4}	{0, 2, 3, 4}	{1, 2, 3}
{1, 2}	{3, 4}	{0, 2}
{3}	{2}	{1}
{1, 2, 3}	{2, 3, 4}	{0, 1, 2}
{3, 4}	{2}	{1, 3}
{2}	{3, 4}	{2}
{2, 3, 4}	{2, 3, 4}	{1, 2, 3}
{0, 1, 2}	{0, 2, 3, 4}	{0, 1, 2}
{1, 3}	{2, 3}	{0, 1, 2}
{2, 3}	{2, 3, 4}	{1, 2}

Après renumérotation la table de l'AFD devient :

Etats	Symboles	
	a	b
A	B	C
B	D	E
C	F	B
D	D	G
E	H	B
F	I	C
G	J	K
H	I	L
I	H	I
J	J	G
K	D	K
L	M	K
M	J	E

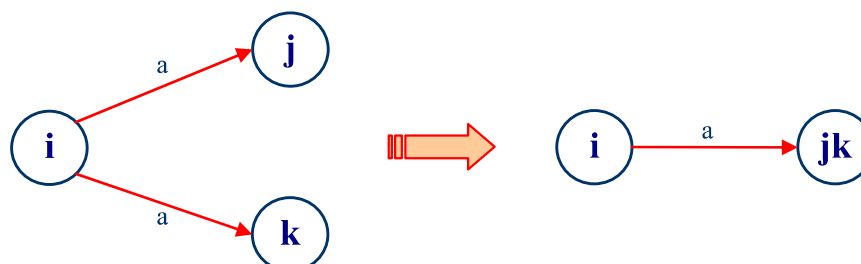
2.6.4.2 Transformation manuelle d'un AFN en AFD (cas général)

On utilise les règles de conversion suivantes pour rendre déterministe un AFN pouvant contenir des ϵ -transitions:

1. Si on peut passer d'un état i à l'état j avec une transition étiquetée ϵ alors les états i et j appartiennent à la même classe.



2. Si l'état i mène à l'état j et l'état k par des transitions identiques (même étiquette), alors les états j et k appartiennent à la même classe.



3. Une classe d'état est finale dans l'AFD si elle contient au moins un état final de l'AFN.

Ainsi, l'AFD équivalent à un AFN original possède des états qui sont des classes (regroupements) d'états de l'automate original. Les nœuds de l'AFD correspondent à des sous ensembles de nœuds de l'AFN qui ne sont pas forcément disjoints.

Si l'AFN à n nœuds, l'AFD peut en avoir 2^n . Donc l'AFD peut être beaucoup plus volumineux que l'AFN équivalent, cependant, en pratique le nombre d'états est souvent plus petit que 2^n .

Exemple

Considérons l'automate non déterministe de la figure 2.9, qui reconnaît l'ensemble des chaînes $\{a, aa, ab, aba, abb\}$.

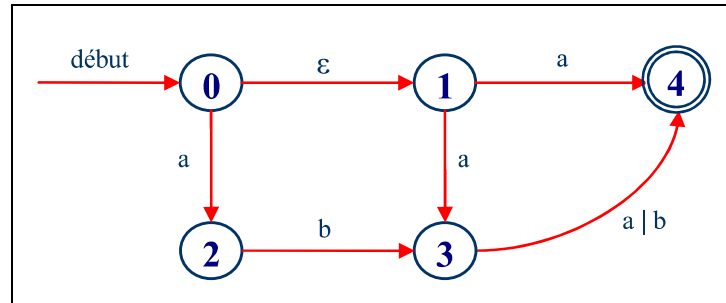


Figure 2.9. Exemple d'AFN

L'AFD correspondant est donné par la figure 2.10, on constate que les états ne sont pas disjoints (234, 34, 4).

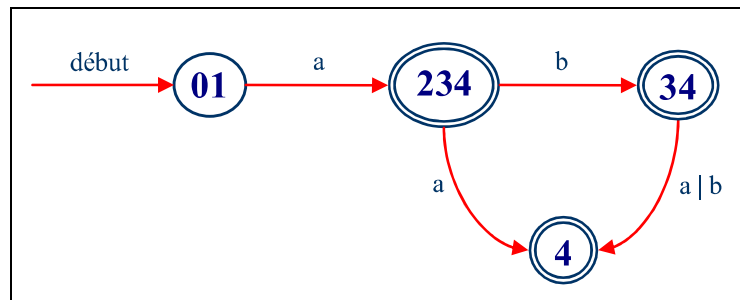


Figure 2.10. AFD équivalent à l'AFN de la figure 2.9

2.6.4.3 Méthode de construction de sous-ensembles pour la transformation des AFN en AFD (cas général)

La construction manuelle décrite précédemment (voir section 2.6.4.2) est simple, mais difficile à appliquer lorsque le nombre d'états est important. Un algorithme plus formel peut être utilisé pour cette étape. Ce dernier est connu sous le nom d'algorithme de *construction de sous-ensembles* et permet de construire une table de transition de l'AFD en se basant sur celle de l'AFN équivalent.

a- Notations et opérations utilisées

Soit un AFN noté N et D son AFD équivalent.

- $DTrans$ est la table des transitions de D
- $Détats$ est l'ensemble des états de D
- e est un état de N , e_0 est l'état initial de N
- T est un ensemble d'états de N
- L'opération ε -fermeture(e) est l'ensemble des états de N accessibles à partir de l'état e par des ε -transitions uniquement.

Remarque

Un état est accessible à partir de lui-même par une ε -transition (même si l'arc n'est pas visible).

- L'opération ε -fermeture(T) donne l'ensemble des états de N accessibles à partir des états e ($e \in T$) par des ε -transitions uniquement.

Dans l'AFN représenté par la figure 2.11, ε -fermeture(1) = $\{1, 2, 3, 4, 8\}$, ε -fermeture(2) = $\{2, 3, 4\}$, ε -fermeture($\{2, 5\}$) = $\{2, 3, 4, 5, 6, 7\}$, ε -fermeture($\{3, 6\}$) = $\{3, 4, 6, 7\}$

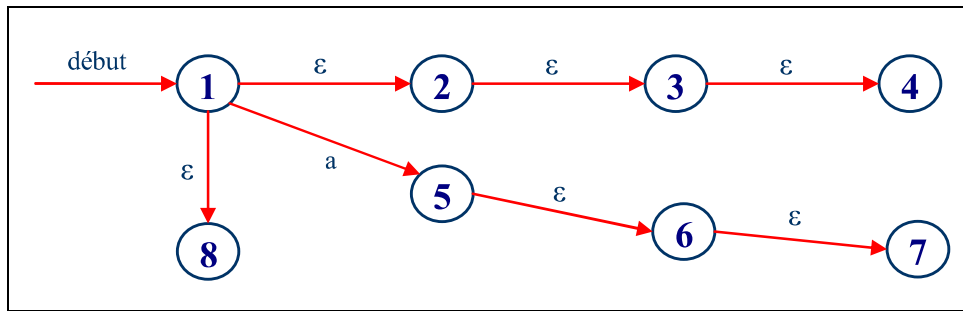


Figure 2.11. Exemple d'AFN pour le calcul ϵ -fermeture

- L'opération $Transiter(T, a)$ donne l'ensemble des états de N vers lesquels il existe une transition avec le symbole d'entrée a à partir des états $e \in T$.

Dans l'AFN représenté par la figure 2.12, $Transiter(\{1, 5\}, a) = \{3, 8, 6\}$, $Transiter(\{1\}, b) = \{7\}$, $Transiter(\{2\}, b) = \{\}$

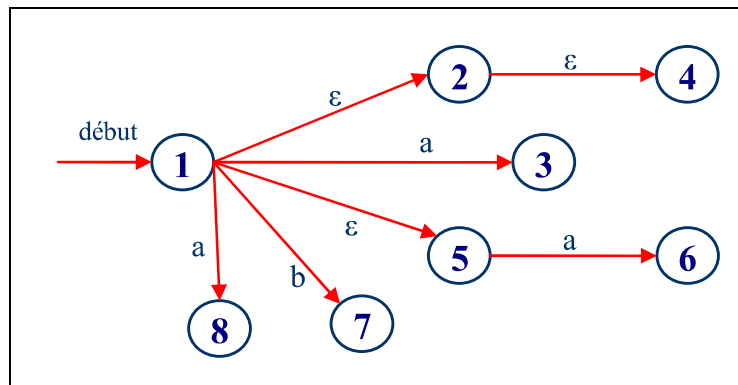


Figure 2.12. Exemple d'AFN pour le calcul de $Transiter$

b- Principe de l'algorithme de construction de sous ensembles

Chaque état de D correspond à un sous ensemble d'états de N de sorte que D simule en parallèle tous les déplacements possibles que N peut effectuer pour une chaîne d'entrée donnée. Ceci signifie que chaque état de D correspond à un ensemble d'états de N dans lesquels N pourrait se trouver après avoir lu une suite donnée de symboles d'entrée, en incluant toutes les ϵ -transitions possibles avant ou après la lecture des symboles.

L'état de départ de D est ϵ -fermeture de e_0 . Un état de D est final s'il correspond à un ensemble d'états de N contenant, au moins, un état final. On ajoute des états et des transitions à D en utilisant l'algorithme de construction de sous ensembles qui se présente comme suit :

Algorithme ConstructionSousEnsembles ;
Début
 ϵ -fermeture(e_0) est l'unique état de Détats et il est non marqué;
Tantque il existe un état non marqué dans Détats **Faire**
 Début
 marquer T ;
 Pour chaque symbole d'entrée a **faire**
 Début
 $U \leftarrow \epsilon$ -fermeture($Transiter(T, a)$) ;
 Si $U \notin$ Détats **Alors** Ajouter U comme nœud non marqué à Détats
 $DTrans[T, a] \leftarrow U$;
 Fin ;
 Fin ;
Fin ;

Exemple : Application de l'algorithme de construction de sous ensemble sur l'AFN obtenu dans la figure 2.8.

$$\epsilon\text{-fermeture}(0) = \{0, 1, 2, 4, 7\} = A$$

Pour T=A

Marquer A

$$\varepsilon\text{-fermeture}(\text{Transiter}(A, a)) = \varepsilon\text{-fermeture}(\{3, 8\}) = \{3, 6, 1, 2, 4, 7, 8\} = \{1, 2, 3, 4, 6, 7, 8\} = B$$

DTran [A, a] ← B

$$\varepsilon\text{-fermeture}(\text{Transiter}(A, b)) = \varepsilon\text{-fermeture}(\{5\}) = \{5, 6, 1, 2, 4, 7\} = \{1, 2, 4, 5, 6, 7\} = C$$

DTran [A, b] ← C

On continue à appliquer l'algorithme avec les états actuellement non marqués B et C.

Pour T=B

Marquer B

$$\varepsilon\text{-fermeture}(\text{Transiter}(B, a)) = \varepsilon\text{-fermeture}(\{3, 8\}) = B$$

DTran [B, a] ← B

$$\varepsilon\text{-fermeture}(\text{Transiter}(B, b)) = \varepsilon\text{-fermeture}(\{5, 9\}) = \{5, 6, 1, 2, 4, 7, 9\} = \{1, 2, 4, 5, 6, 7, 9\} = D$$

DTran [B, b] ← D

Pour T=C

Marquer C

$$\varepsilon\text{-fermeture}(\text{Transiter}(C, a)) = \varepsilon\text{-fermeture}(\{3, 8\}) = B$$

DTran [C, a] ← B

$$\varepsilon\text{-fermeture}(\text{Transiter}(C, b)) = \varepsilon\text{-fermeture}(\{5\}) = \{5, 6, 1, 2, 4, 7\} = \{1, 2, 4, 5, 6, 7\} = C$$

DTran [C, b] ← C

On continue à appliquer l'algorithme avec le seul état actuellement non marqué D.

Pour T=D

Marquer D

$$\varepsilon\text{-fermeture}(\text{Transiter}(D, a)) = \varepsilon\text{-fermeture}(\{3, 8\}) = B$$

DTran [D, a] ← B

$$\varepsilon\text{-fermeture}(\text{Transiter}(D, b)) = \varepsilon\text{-fermeture}(\{5, 10\}) = \{5, 6, 1, 2, 4, 7, 10\} = \{1, 2, 4, 5, 6, 7, 10\} = E$$

DTran [D, b] ← E

On continue à appliquer l'algorithme avec le seul état actuellement non marqué E.

Pour T=E

Marquer E

$$\varepsilon\text{-fermeture}(\text{Transiter}(E, a)) = \varepsilon\text{-fermeture}(\{3, 8\}) = B$$

DTran [E, a] ← B

$$\varepsilon\text{-fermeture}(\text{Transiter}(E, b)) = \varepsilon\text{-fermeture}(\{5\}) = C$$

DTran [E, b] ← C

Après plusieurs itérations, on arrive ainsi à un point où tous les sous ensembles d'états de l'AFD sont marqués. A la fin les cinq ensembles d'états construits sont :

- $A = \{0, 1, 2, 4, 7\}$
- $B = \{1, 2, 3, 4, 6, 7, 8\}$
- $C = \{1, 2, 4, 5, 6, 7\}$
- $D = \{1, 2, 4, 5, 6, 7, 9\}$
- $E = \{1, 2, 4, 5, 6, 7, 10\}$

A est l'état initial de l'automate et E son état final.

La table DTrans de l'AFD obtenu après l'application de l'algorithme de sous-ensembles à l'AFN de la figure 2.8., est donnée par :

Etats	Symboles	
	a	b
A	B	C
B	B	D
C	B	C
D	B	E
E	B	C

La figure 2.13 donne le graphe de transition de l'AFD obtenu :

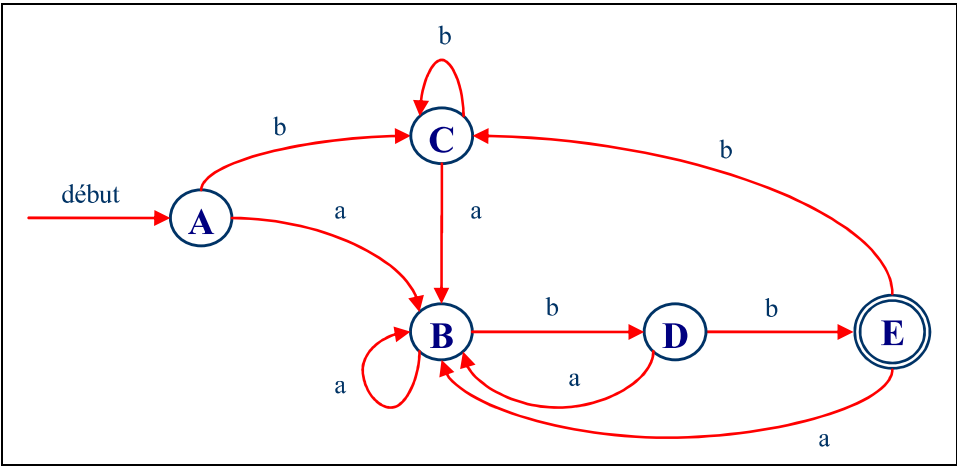


Figure 2.13. AFD équivalent à l'AFN de la figure 2.8

c- Autre formulation de l'algorithme de construction de sous ensembles

L'algorithme de construction de sous-ensembles peut être formulé d'une manière plus condensée en utilisant une représentation tabulaire. Notons que le principe est toujours le même que dans la première formulation présentée précédemment.

- 1- Commencer avec la ϵ -fermeture de l'état initial (elle représente le nouvel état initial) ;
2- Rajouter dans la table de transition toutes les ϵ -fermetures des nouveaux états produits avec leurs transitions ;
3- Recommencer l'étape 2 jusqu'à ce qu'il n'y ait plus de nouvel état ;
4- Tous les ϵ -fermetures contenant au moins un état final du premier automate deviennent finaux ;
5- Renommer les états en tant qu'états simples.

Exemple : Application de cette formulation de l'algorithme de construction de sous ensemble sur l'AFN obtenu dans la figure 2.8.

La table de transition de l'AFD obtenu est donnée par :

Etats	Symboles	
	a	b
0,1,2,4,7	1,2,3,4,6,7,8	1,2,4,5,6, 7
1,2,3,4,6,7,8	1,2,3,4,6,7,8	1,2,4,5,6,7,9
1,2,4,5,6, 7	1,2,3,4,6,7,8	1,2,4,5,6, 7
1,2,4,5,6,7,9	1,2,3,4,6,7,8	1,2,4,5,6,7,10
1,2,4,5,6,7,10	1,2,3,4,6,7,8	1,2,4,5,6, 7

Après renumérotation des états, nous obtenons la table suivante qui est identique à celle obtenue avec la première formulation de l'algorithme de construction de sous ensembles (voir juste avant la figure 2.13) :

Etats	Symboles	
	a	b
A	B	C
B	B	D
C	B	C
D	B	E
E	B	C

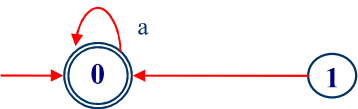
2.6.5 Minimisation d'un AFD (étape 5)

Intuitivement, la minimisation d'un AFD consiste en deux opérations principales :

- Elimination de tous les états inaccessibles (et improductifs) de l'AFD
- Regroupement des états équivalents en les remplaçant par des classes d'équivalence d'états

Sachant que :

- Un état e est accessible s'il existe une chaîne menant à e à partir de l'état initial. Un état, autre que l'état initial, est inaccessible lorsqu'aucun arc n'arrive sur cet état dans un chemin provenant de l'état initial.

Par exemple, dans l'automate  l'état 1 est inaccessible.

- On dit que deux états sont équivalents si ces états permettent d'atteindre un état final à travers la même chaîne. En d'autres termes, tous les suffixes reconnus à partir de ses états sont exactement les mêmes.

Par exemple, la figure 2.14 montre la minimisation d'un automate contenant deux états équivalents (états 3 et 4).

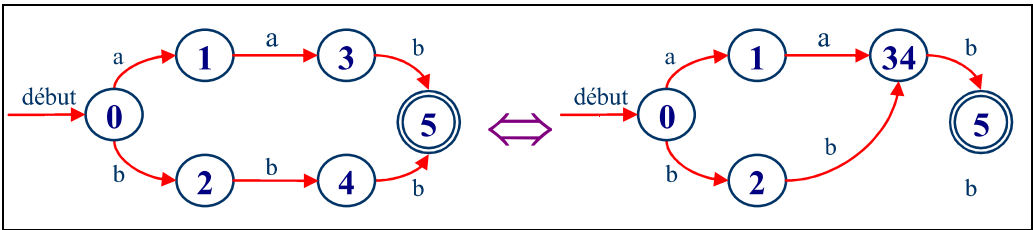


Figure 2.14. Exemple de minimisation d'un AFD

Un algorithme simplifié de minimisation peut être utilisé, il fonctionne par raffinements successifs en définissant des classes d'équivalence d'états qui vont correspondre aux états du nouvel automate (minimisé). Cet algorithme comprend les étapes suivantes :

1. Définir deux classes, C1 contenant les états finaux et C2 les états non finaux.
2. S'il existe un symbole a et deux états e_1 et e_2 d'une même classe tels que $\text{Transiter}(e_1, a)$ et $\text{Transiter}(e_2, a)$ n'appartiennent pas à la même classe (d'arrivée), alors créer une nouvelle classe et séparer e_1 et e_2 . On laisse dans la même classe tous les états qui donnent un état d'arrivée dans la même classe.
3. Recommencer l'étape 2 jusqu'à ce qu'il n'y ait plus de classes à séparer.
4. Chaque classe restante forme un état du nouvel automate.

2.6.6 Simulation d'un AFD (étape 6)

Soit une chaîne x représentée par un fichier de caractères se terminant par EoF qui est un caractère spécial indiquant la fin d'un fichier. Soit un AFD D avec un état initial e_0 et un ensemble d'état finaux F . Soit la fonction $\text{Transiter}(e, ch)$ qui donne l'état vers lequel il y a transition à partir de l'état e avec le caractère ch .

L'algorithme de simulation de l'AFD est le suivant :

```
Algorithme SimulAFD ;  
Début  
  e ← e0 ;  
  carsuiv(ch) ;  
  Tantque ch ≠ EoF Faire  
    Début  
      e ← Transiter(e, ch) ;  
      carsuiv(ch) ;  
    Fin ;  
    Si e ∈ F Alors accepter(x)  
    Sinon rejeter(x) ;  
Fin ;
```

2.7 Générateurs d'analyseurs lexicaux

Les étapes de construction d'un analyseur lexical à partir de la spécification des unités lexicales peuvent être automatisées en utilisant un outil logiciel dit générateur d'analyseur lexical tel que l'outil LEX qui peut être considéré comme le plus ancien. Cette approche de construction des analyseurs lexicaux est considérée comme la plus simple car on se limite à fournir les définitions exactes des unités lexicales du langage considéré et le générateur fait le reste en générant le code source de l'analyseur lexical.

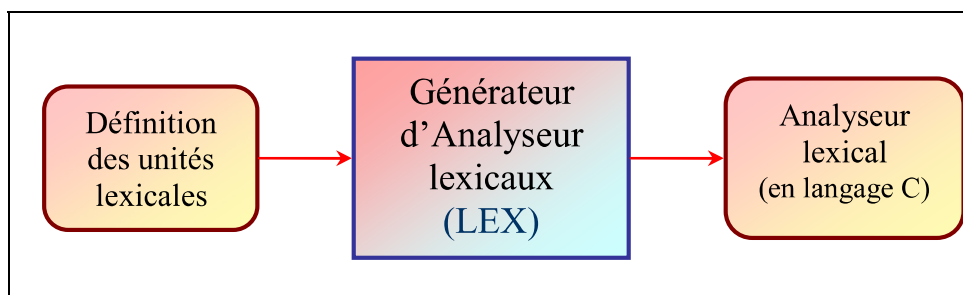


Figure 2.15. Construction d'un analyseur lexical en utilisant un générateur

Le générateur LEX permet de générer un analyseur lexical en langage C à partir des spécifications des unités lexicales exprimées en langage LEX. Le noyau de LEX permet de produire l'automate d'état fini correspondant à la reconnaissance de l'ensemble des unités lexicales spécifiées. Ainsi, LEX construit une table de transition pour un automate à états finis à partir des modèles d'expressions régulières de la spécification en langage LEX.

L'analyseur lexical lui-même consiste en un simulateur d'automate à états finis (en langage C), qui utilise la table de transition obtenue pour rechercher les unités lexicales dans le texte en entrée. La spécification des unités lexicales est effectuée en utilisant le langage LEX, en créant un programme */ex/* constitué de trois parties :

```
Déclarations  
%%  
Règles de traduction  
%%  
Procédures auxiliaires
```

La première partie contient des déclarations de variables, de constantes et des définitions régulières utilisées comme composantes des expressions régulières qui apparaissent dans les règles de traduction.

Les règles de traduction sont des instructions (en langage C) de la forme $m_i \{action_i\}$ où chaque m_i est une expression régulière et chaque $action_i$ est un fragment de programme qui décrit quelle action l'analyseur lexical devrait réaliser quand une unité lexicale concorde avec le modèle m_i .

Les procédures auxiliaires sont toutes les procédures qui pourraient être utiles dans les actions.

SERIE DE TD N°:1 COMPILATION

ANALYSE LEXICALE

Exercice 1

Donner la définition régulière et le diagramme de transition correspondant à des identificateurs qui ne dépassent pas 4 caractères (lettres ou chiffres) dont le premier est obligatoirement une lettre.

Proposer une implémentation pour le diagramme de transition obtenu (algorithme correspondant à l'analyseur de ce type d'unités lexicales).

Exercice 2

Soit un automate d'états finis dont l'ensemble des états est $\{0, 1, 2, 3, 4\}$, état initial: 0, états finaux: 0 et 3, le vocabulaire étant $\{a, b\}$. Notons T la fonction de transition de cet automate.

$T(0, a) = \{2\}$, $T(0, b) = \{1\}$, $T(1, a) = \{2\}$, $T(1, b) = \{1\}$, $T(2, a) = \{3\}$, $T(2, b) = \{2\}$,

$T(3, a) = \{2, 4\}$, $T(3, b) = \{1, 3\}$, $T(4, a) = \{4\}$, $T(4, b) = \{4\}$

1. Etablir la table de transition et la représentation graphique de l'automate.
2. Etablir la table de transition et la représentation graphique de l'automate déterministe équivalent.
3. Donner le chemin de reconnaissance des chaînes *bababb* et *abb* par l'automate déterministe obtenu.
4. Donner le chemin de reconnaissance de la chaîne *aaab* par l'automate initial puis proposer une minimisation de cet automate qui n'affecte pas le langage qu'il reconnaît.

Exercice 3

Effectuer la minimisation des automates suivants :

- 1). L'AFD de la figure 2.13 (page 23 du support de cours).
- 2). L'AFD obtenu dans la table en page 17 du support de cours (états de A à M).

Exercice 4

Soient les expressions régulières suivantes: *a*, *abb*, a^*b^+

- 1). Appliquer les étapes 2 à 5 de la démarche de construction d'un analyseur lexical basée sur les automates d'états finis, afin de construire un analyseur lexical pour les trois expressions régulières précédentes, correspondant à des unités lexicales.
- 2). Décrire l'analyse lexicale des chaînes *aaba* et *abb* en donnant leur chemin de reconnaissance (utiliser l'algorithme de simulation d'un AFD qui représente la 6^{ème} étape de construction de l'analyseur).

Exercice 5

Soit un langage, représentant un petit sous-ensemble du langage Pascal, et constitué par les unités lexicales suivantes :

- Des identificateurs commençant par une lettre suivie d'une combinaison quelconque de lettres ou de chiffres
- Des constantes numériques entières non signées (sans limitation de longueur)
- L'affectation (:=)

Construire l'analyseur lexical correspondant à ce langage en utilisant l'approche basée sur les automates d'états finis.

CHAPITRE 3 : ANALYSE SYNTAXIQUE : INTRODUCTION, RAPPELS ET COMPLEMENTS

3.1 Rôle d'un analyseur syntaxique

L'analyseur syntaxique (Parser en anglais) a comme rôle l'analyse des séquences d'unités lexicales, obtenues par l'analyseur lexical, conformément à une grammaire qui engendre le langage considéré.

L'analyseur syntaxique reconnaît la structure syntaxique d'un programme source et produit une représentation qui est généralement sous forme d'un arbre syntaxique. Ce dernier peut être décoré par des informations sémantiques puis utilisé pour produire un programme cible.

Pour les langages simples, l'analyseur syntaxique peut ne pas livrer une représentation explicite de la structure syntaxique du programme, mais remplit le rôle d'un module principal qui appelle des procédures d'analyse lexicale, d'analyse sémantique et de génération de code.

Ainsi, les principales fonctions d'un analyseur syntaxique peuvent être résumées comme suit :

1. L'analyse de la chaîne des unités lexicales délivrées par l'analyseur lexical, pour vérifier si cette chaîne peut être engendrée par la grammaire du langage considéré.
2. La détection des erreurs syntaxiques si la syntaxe du langage n'est pas respectée.
3. La construction éventuelle d'une représentation interne qui sera utilisée par les phases ultérieures.

3.2 Approches d'analyse syntaxique

Il existe trois grandes catégories de méthodes pour l'analyse syntaxique :

1. Méthodes universelles : Elles sont généralement tabulaires comme celles de Cocke-Younger-Kasami (1965-1967) et de Earley (1970) qui peuvent analyser une grammaire quelconque. Cependant, ces méthodes sont trop inefficaces pour être utilisées dans les compilateurs industriels.
2. Méthodes descendantes : Ces méthodes sont dites aussi **Top-Down** car elles construisent des arbres d'analyse de haut en bas (de la racine aux feuilles).
3. Méthodes ascendantes : Ces méthodes sont dites aussi **Bottom-up** car elles construisent des arbres d'analyse de bas en haut (remontent des feuilles à la racine).

Exemple d'analyse

Soit la grammaire $G = \{V_T, V_N, S, P\}$

P étant défini par les règles suivantes :

$$\begin{aligned} S &\rightarrow B \\ B &\rightarrow R \mid (B) \\ R &\rightarrow E = E \\ E &\rightarrow a \mid b \mid (E+E) \end{aligned}$$

$V_T = \{a, b, (,), =, +\}$

$V_N = \{S, B, R, E\}$

L'analyse de la chaîne ($a = (b + a)$) par une approche descendante débute par l'axiome et s'effectue par dérivations successives comme suit (si on dérive à partir de la gauche) :

$S \rightarrow B$
 $\rightarrow (B)$
 $\rightarrow (R)$
 $\rightarrow (E = E)$
 $\rightarrow (a = E)$
 $\rightarrow (a = (E + E))$
 $\rightarrow (a = (b + E))$
 $\rightarrow (a = (b + a))$

L'analyse de la chaîne ($a = (b + a)$) par une approche ascendante débute au niveau de la chaîne elle-même (**lire de bas en haut**) et remplace, à chaque étape, une partie de la chaîne par un non-terminal, par réductions successives, comme suit (si on réduit à partir de la gauche) :

S
 B
 (B)
 (R)
 $(E = E)$
 $(E = (E + E))$
 $(E = (E + a))$
 $(E = (b + a))$
 $(a = (b + a))$

Les méthodes ascendantes et descendantes sont utilisées fréquemment pour la mise en oeuvre de compilateurs. Dans les deux cas, l'entrée de l'analyseur syntaxique est parcourue de la gauche vers la droite, un symbole à la fois. Les méthodes ascendantes et descendantes les plus efficaces fonctionnent uniquement avec des sous-classes de grammaires telles que les grammaires LL et LR (qui seront étudiées aux chapitres 4 et 5) qui sont suffisamment expressives pour décrire la majorité des structures syntaxiques des langages de programmation.

Les analyseurs syntaxiques mis en oeuvre manuellement utilisent généralement des méthodes descendantes et des grammaires LL. C'est le cas, par exemple, pour l'analyse prédictive et la descente récursive.

Les méthodes ascendantes sont souvent utilisées par les outils de construction des analyseurs syntaxiques. C'est le cas avec les analyseurs LR qui sont des analyseurs plus généraux que les analyseurs descendants.

La construction d'analyseurs LR est courante dans les environnements de développement de compilateurs. Notons qu'il existe aussi d'autres méthodes ascendantes telles que la précédence d'opérateurs, la précédence simple, la précédence faible et les matrices de transition.

3.3 Concepts de base

3.3.1 Notion de grammaire

Une grammaire G est un formalisme qui permet la génération des chaînes d'un langage. Elle est formellement définie par un quadruplet $G = (V_T, V_N, S, P)$ où

- V_T est le vocabulaire terminal contenant l'ensemble des symboles atomiques individuels, pouvant être composés en séquence pour former des phrases et souvent notés en lettres minuscules.

- V_N est le vocabulaire non terminal contenant l'ensemble des symboles non terminaux représentant des constructions syntaxiques. Ces symboles sont souvent notés en majuscules.
- S est appelé axiome ou symbole initial de la grammaire.
- P est l'ensemble des règles de production de la forme $\alpha \rightarrow \beta$ où α et β appartiennent à $(V_T \cup V_N)^*$

Selon la forme des règles de production, il existe quatre types de grammaires que nous présentons selon la classification de Chomsky. Dans la suite nous considérons que si V est un vocabulaire alors V^* est l'ensemble de toutes les chaînes résultant de la concaténation des éléments de V .

Type de grammaire	Forme des règles de production
Type 0 (Grammaires sans restrictions)	$\alpha \rightarrow \beta$ avec $\alpha \in (V_T \cup V_N)^+$ et $\beta \in (V_T \cup V_N)^*$ Autrement dit il n'y a aucune restriction sur α et β
Type 1 (Grammaires à contexte lié ou Contextuelles)	$\alpha \rightarrow \beta$ avec $\alpha \in (V_T \cup V_N)^+$ et $\beta \in (V_T \cup V_N)^*$ et $ \alpha \leq \beta $ et ϵ (la chaîne vide) ne peut être généré que par l'axiome avec la règle $S \rightarrow \epsilon$
Type 2 (Grammaires à contexte libre, non contextuelles, algébriques ou encore de Chomsky)	$A \rightarrow \alpha$ avec $A \in V_N$ et $\alpha \in (V_T \cup V_N)^*$
Type 3 (Grammaires régulières ou linéaires droites)	$A \rightarrow \alpha B$ ou $A \rightarrow \alpha$ avec $A, B \in V_N$ et $\alpha \in V_T^*$

3.3.2 Expression des grammaires

A part l'énumération des règles de production, il est possible de définir une grammaire en utilisant des notations textuelles telles que la **forme BNF** ou la **forme EBNF** ainsi que des représentations graphiques telles que les **diagrammes syntaxiques**.

a- La notation BNF (Backus-Naur Form)

Cette notation est due aux créateurs de Fortran (J. Backus, 1955) et Algol (P. Naur, 1963). Les règles sont écrites avec les conventions suivantes :

- Un symbole $A \in V_N$ est noté par **<A>** ou **A** (en utilise des lettres majuscules)
- Un symbole $a \in V_T$ est noté sans les **< >** ou encore par **"a"** ou **a** (en utilise des lettres minuscules)
- Le signe $\rightarrow$ devient **::=**
- Un ensemble de règles $A \rightarrow \alpha, A \rightarrow \beta, \dots, A \rightarrow \gamma$ sera remplacé par **$A \rightarrow \alpha | \beta | \dots | \gamma$**

Exemple

<Bloc> ::= **Begin <List Opt Inst> **End** .**

<List Opt Inst> ::= <List Inst> | ϵ

<List Inst> ::= <List Inst>; <Inst> | <Inst>

<Inst> ::= **id := <Exp> | **if** <Exp> **then** <Inst> | **if** <Exp> **then** <Inst> **else** <Inst>**

b- La notation EBNF (Extended Backus-Naur Form)

C'est une représentation plus réduite que BNF qui permet d'écrire les grammaires sous une forme plus condensée en utilisant les signes **{}** et **[]**.

- Une partie optionnelle est notée entre **[]** (apparaît 0 ou 1 fois)
- Une partie qui peut apparaître de façon répétée est notée entre **{}** (apparaît 0 ou N fois)

Notons que, dans les notations BNF et EBNF, les parenthèses peuvent être utilisées pour éviter les ambiguïtés, comme c'est le cas dans la règle suivante :

$\langle \text{Inst} \rangle ::= \text{For } \text{id} := \langle \text{Exp} \rangle (\text{to} \mid \text{downto}) \langle \text{Exp} \rangle \text{ do } \langle \text{Bloc} \rangle$

la partie (**to** | **downto**) exprime un choix entre **to** et **downto**

Exemple

L'exemple présenté avec la notation BNF peut être noté comme suit en utilisant EBNF :

$\langle \text{Bloc} \rangle ::= \text{Begin } [\langle \text{List Inst} \rangle] \text{ End } .$

$\langle \text{List Inst} \rangle ::= \langle \text{Inst} \rangle \{ ; \langle \text{Inst} \rangle \}$

$\langle \text{Inst} \rangle ::= \text{id} := \langle \text{Exp} \rangle \mid \text{if } \langle \text{Exp} \rangle \text{ then } \langle \text{Inst} \rangle [\text{else } \langle \text{Inst} \rangle]$

Cette représentation est la même que la suivante :

$\langle \text{Bloc} \rangle ::= \text{Begin } [\langle \text{Inst} \rangle \{ ; \langle \text{Inst} \rangle \}] \text{ End } .$

$\langle \text{Inst} \rangle ::= \text{id} := \langle \text{Exp} \rangle \mid \text{if } \langle \text{Exp} \rangle \text{ then } \langle \text{Inst} \rangle [\text{else } \langle \text{Inst} \rangle]$

c- Les diagrammes syntaxiques

C'est une représentation sous forme graphique des règles du langage où les terminaux sont **encerclés** et les non terminaux sont **encadrés**. La figure 3.1 donne les diagrammes syntaxiques correspondant à l'exemple précédent.

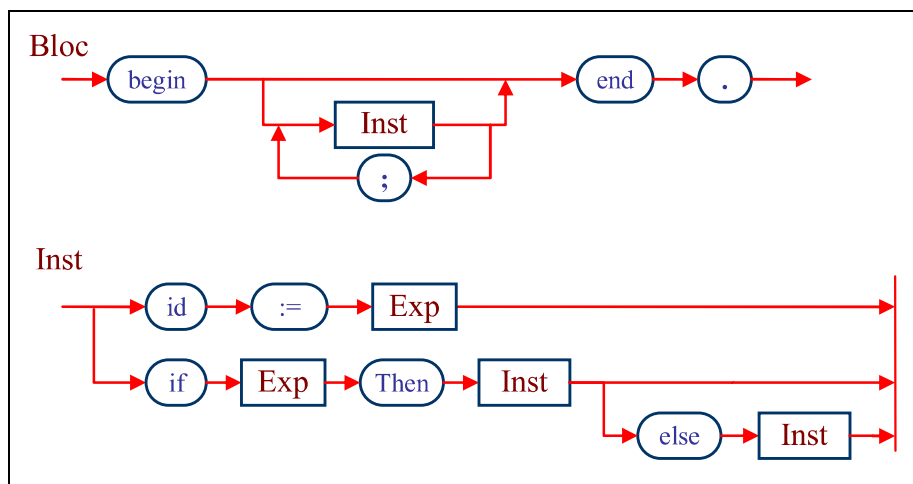


Figure 3.1. Exemple de diagrammes syntaxiques

3.3.3 Dérivation et arbres syntaxiques

a- Dérivation

On appelle dérivation, l'application d'une ou plusieurs règles à partir d'une phrase (chaîne) de $(V_T \cup V_N)^+$.

On note $\rightarrow$ une dérivation obtenue par l'application d'une seule règle de production et on ^{*}note $\rightarrow$ une dérivation obtenue par l'application de plusieurs règles de production.

Exemple

Soit les règles :

$S \rightarrow A$

$A \rightarrow bB$

$$B \rightarrow c$$

La chaîne bc peut être dérivée par l'application de ces règles comme suit :

$$\begin{aligned} S &\rightarrow A \\ &\rightarrow bB \\ &\rightarrow bc \end{aligned}$$

On peut noter ces dérivations par une expression plus condensée $S \xrightarrow{*} bc$

D'une manière générale, si G est une grammaire définie par (V_T, V_N, S, P) alors le langage généré peut être noté par : $L(G) = \{x \in V_T^* \mid S \xrightarrow{*} x\}$.

On distingue deux façons d'effectuer une dérivation :

- Dérivation gauche (la plus à gauche ou « left most ») : Elle consiste à remplacer toujours le symbole non terminal le plus à gauche.
- Dérivation droite (la plus à droite ou « right most ») : Elle consiste à remplacer toujours le symbole non terminal le plus à droite.

Exemple

Soit $G = (\{a, +, (,)\}, \{E, F\}, E, P)$

P est défini par :

$$\begin{aligned} E &\rightarrow E + F \\ E &\rightarrow F \\ F &\rightarrow (F) \mid a \end{aligned}$$

Peut-on affirmer que a+a appartient au langage L(G) ?

Par dérivations gauches, on a :

$$E \rightarrow E + F \rightarrow F + F \rightarrow a + F \rightarrow a + a \quad \text{donc} \quad E \xrightarrow{*} a+a \quad \text{et la chaîne appartient au langage}$$

Par dérivations droites, on a :

$$E \rightarrow E + F \rightarrow E + a \rightarrow F + a \rightarrow a + a \quad \text{donc} \quad E \xrightarrow{*} a+a \quad \text{et la chaîne appartient au langage}$$

b- Arbre de dérivation (Arbre syntaxique)

On appelle **arbre de dérivation** ou **arbre syntaxique concret** un arbre tel que

1. La racine est l'axiome de la grammaire
2. Les feuilles de l'arbre sont des symboles terminaux
3. Les nœuds sont les symboles non terminaux
4. Pour un nœud interne d'étiquette A, le mot $\alpha_1\alpha_2 \dots \alpha_n$ obtenu en lisant de gauche à droite les étiquettes des nœuds fils de A correspond à une règle de la grammaire $A \rightarrow \alpha_1\alpha_2 \dots \alpha_n$
5. Le mot m, dont on fait l'analyse, est constitué des étiquettes des feuilles, lues de gauche à droite

Exemple

Soit $G = (\{a, b, c\}, \{S, T\}, S, P)$

P est défini par :

$$\begin{aligned} S &\rightarrow aTb \mid c \\ T &\rightarrow cSS \mid S \end{aligned}$$

L'arbre de dérivation de accacbb est donné par la figure 3.2.

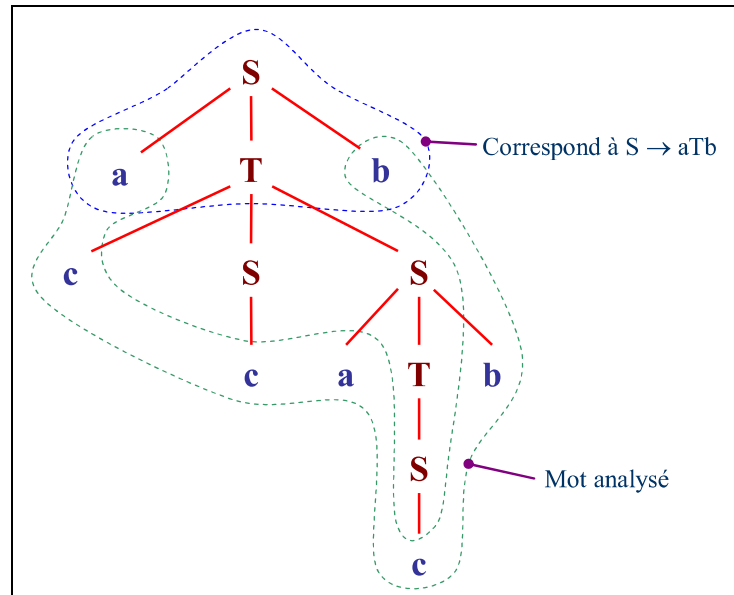


Figure 3.2. Exemple d'arbre de dérivation

c- Arbre abstrait

On appelle **arbre syntaxique abstrait** ou **arbre abstrait** une forme condensée d'arbre syntaxique qui est utilisée dans les compilateurs. L'arbre abstrait est obtenu par des transformations simples de l'arbre de dérivation.

L'arbre syntaxique abstrait est plus compact que l'arbre syntaxique concret et contient des informations sur la suite des actions effectuées par un programme. Dans un arbre abstrait, les opérateurs et les mots clés apparaissent comme des nœuds intérieurs et les opérandes apparaissent comme des feuilles.

Exemple

Si on considère, la grammaire $G = (\{a, +, (,)\}, \{E, T, F\}, E, P)$

P étant défini par :

$E \rightarrow T \mid E + T$

$T \rightarrow F \mid T * F$

$F \rightarrow a \mid (E)$

Les arbres syntaxiques (concret et abstrait) pour la chaîne $a + a * a$ sont donnés par la figure 3.5.

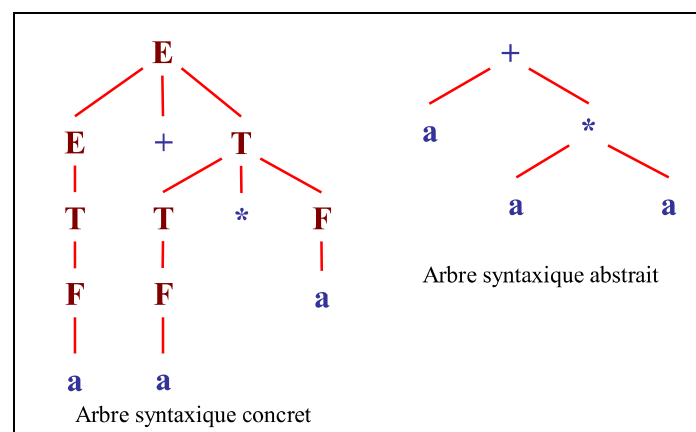


Figure 3.4. Arbre syntaxique concret et abstrait pour la chaîne $a + a * a$

d- Grammaire ambiguë

On dit qu'une grammaire est ambiguë si elle produit plus d'un arbre syntaxique pour une chaîne donnée.

Si on considère la grammaire $G = (\{+, -, *, /, id\}, \{E\}, E, P)$ où P est défini par :

$$E \rightarrow E + E \mid E - E \mid E * E \mid E / E \mid (E) \mid -E \mid id$$

La figure 3.3 donne deux arbres de dérivation pour la chaîne $id + id * id$

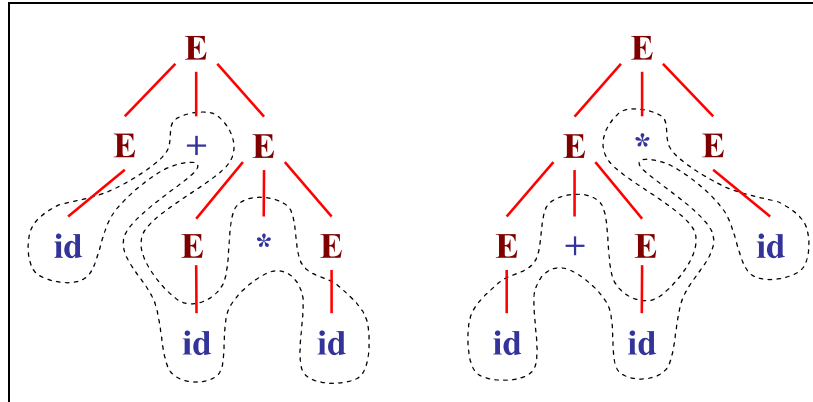


Figure 3.3. Deux arbres de dérivation pour la chaîne $id + id * id$

L'élimination de l'ambiguïté est une tâche importante qu'on exige généralement pour les langages de programmation qui sont basés sur des grammaires non contextuelles. L'élimination de l'ambiguïté peut avoir lieu de deux façons :

1. Élimination de l'ambiguïté par ajout de règles. Dans cette approche, la grammaire reste inchangée, on ajoute, à la sortie de l'analyseur syntaxique, quelques règles dites de désambiguïté, qui éliminent les arbres syntaxiques non désirables et conservent un arbre unique pour chaque chaîne. On peut, par exemple, éliminer l'ambiguïté en affectant des priorités aux opérateurs arithmétiques. Dans les expressions arithmétiques, la priorité est affectée aux parenthèses, puis au signe moins unaire, puis à la multiplication/division et enfin à l'addition/soustraction.

2. Réécriture de la grammaire. Il s'agit de changer la grammaire en incorporant des règles de priorité et d'associativité. Autrement dit, on ajoute de nouvelles règles et de nouveaux symboles non terminaux. La grammaire ambiguë donnée précédemment peut être désambiguïsée comme suit :

$G = (\{+, -, *, /, id\}, \{E, T, F, L\}, E, P)$ où P est défini par :

$$E \rightarrow E + T \mid E - T \mid T$$

$$T \rightarrow T * F \mid T / F \mid F$$

$$F \rightarrow -F \mid L$$

$$L \rightarrow (E) \mid id$$

La chaîne $id + id * id$ a maintenant une seule séquence de dérivations (gauches) :

$$E \rightarrow E + T \rightarrow T + T \rightarrow F + T \rightarrow L + T \rightarrow id + T \rightarrow id + T * F$$

$$\rightarrow id + F * F \rightarrow id + L * F \rightarrow id + id * F \rightarrow id + id * L \rightarrow id + id * id$$

et l'unique arbre syntaxique donné par la figure 3.4.

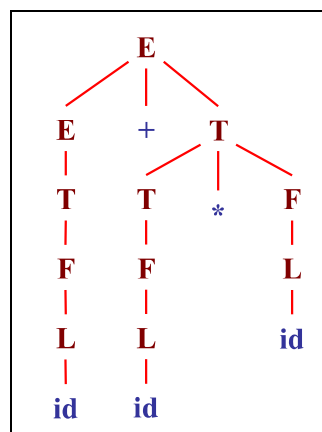


Figure 3.4. Arbre syntaxique de $id + id * id$

SERIE DE TD N°:2 COMPILATION

ANALYSE SYNTAXIQUE : INTRODUCTION, RAPPELS ET COMPLEMENTS

Exercice 1

1) Soit la grammaire $G = (\{a, b\}, \{S, A, B, C\}, S, P)$ où P est défini par :

$S \rightarrow AB \mid BC$

$A \rightarrow BA \mid a$

$B \rightarrow CC \mid b$

$C \rightarrow AB \mid a$

Analyser la chaîne **babaab** de manière descendante puis ascendante, en construisant, à chaque fois, son arbre de dérivation.

Exercice 2

1) Soit la grammaire des expressions arithmétiques $G = (\{+, -, *, /, a, b, c, (,)\}, \{E\}, E, P)$ où P est défini par :

$E \rightarrow E + E \mid E - E \mid E * E \mid E / E \mid (E) \mid -E \mid a \mid b \mid c$

Donner l'arbre de dérivation de la chaîne **b + a * b - c** ? Que peut-on en déduire ?

2) Soit la grammaire $G' = (\{+, -, *, /, a, b, (,)\}, \{E, T, F, L\}, E, P)$ où P est défini par :

$E \rightarrow E + T \mid E - T \mid T$

$T \rightarrow T * F \mid T / F \mid F$

$F \rightarrow -F \mid L$

$L \rightarrow (E) \mid a \mid b \mid c$

a) Donner l'arbre de dérivation de la chaîne **b + a * b - c**. Que peut-on en déduire ?

b) Donner l'arbre abstrait correspondant à la chaîne **b + a * b - c**.

c) Donner les arbres syntaxiques concret et abstrait de la chaîne **b - a + c**.

Exercice 3

1) Soit la grammaire des expressions booléennes $G = (\{\text{ou}, \text{et}, \text{non}, \text{vrai}, \text{faux}, (,)\}, \{A\}, A, P)$ où P est défini par :

$A \rightarrow A \text{ ou } A \mid A \text{ et } A \mid \text{non } A \mid (A) \mid \text{vrai} \mid \text{faux}$

Donner l'arbre de dérivation de la chaîne **non faux ou vrai et vrai** ? Que peut-on en déduire ?

2) Soit la grammaire $G' = (\{\text{ou}, \text{et}, \text{non}, \text{vrai}, \text{faux}, (,)\}, \{A, B, C, D\}, A, P)$ où P est défini par :

$A \rightarrow A \text{ ou } B \mid B$

$B \rightarrow B \text{ et } C \mid C$

$C \rightarrow \text{non } C \mid D$

$D \rightarrow (A) \mid \text{vrai} \mid \text{faux}$

Donner l'arbre de dérivation de la chaîne **non faux ou vrai et vrai** ? Conclusion ?

Exercice 4

Soit la grammaire G d'un langage proche de Pascal, exprimée sous forme EBNF de la manière suivante, exprimer la grammaire G sous forme de diagrammes syntaxiques.

$\langle \text{Programme} \rangle ::= \underline{\text{Program}} \text{ ident} ; \langle \text{Bloc} \rangle .$

$\langle \text{Bloc} \rangle ::= [\underline{\text{Const}} \langle \text{SuitConst} \rangle ;] [\underline{\text{Var}} \langle \text{SuitVar} \rangle ;] \{ \underline{\text{Procédure}} \text{ ident} ; \langle \text{Bloc} \rangle ; \}$
 $\quad \underline{\text{Begin}} \langle \text{Inst} \rangle \{ ; \langle \text{Inst} \rangle \} \underline{\text{End}}$

$\langle \text{SuitConst} \rangle ::= \langle \text{DecConst} \rangle \{ , \langle \text{DecConst} \rangle \}$

$\langle \text{DecConst} \rangle ::= \text{ident} = \text{nbEnt}$

$\langle \text{SuitVar} \rangle ::= \text{ident} \{ , \text{ident} \}$

$\langle \text{Inst} \rangle ::= \text{ident} := \langle \text{Exp} \rangle \mid \underline{\text{If}} \langle \text{Cond} \rangle \underline{\text{Then}} \langle \text{Inst} \rangle [\underline{\text{Else}} \langle \text{Inst} \rangle]$

$\quad \mid \underline{\text{Repeat}} \langle \text{Inst} \rangle \underline{\text{Until}} \langle \text{Cond} \rangle \mid \underline{\text{While}} \langle \text{Cond} \rangle \underline{\text{Do}} \langle \text{Inst} \rangle \mid \underline{\text{Begin}} \langle \text{Inst} \rangle \{ ; \langle \text{Inst} \rangle \} \underline{\text{End}}$

$\langle \text{Cond} \rangle ::= \langle \text{Exp} \rangle (= \mid < \mid > \mid < = \mid > =) \langle \text{Exp} \rangle$

$\langle \text{Exp} \rangle ::= \langle \text{Terme} \rangle \{ (+ \mid -) \langle \text{Terme} \rangle \}$

$\langle \text{Terme} \rangle ::= \langle \text{Facteur} \rangle \{ (* \mid /) \langle \text{Facteur} \rangle \}$

$\langle \text{Facteur} \rangle ::= \text{ident} \mid \text{nbEnt} \mid (\langle \text{Exp} \rangle)$